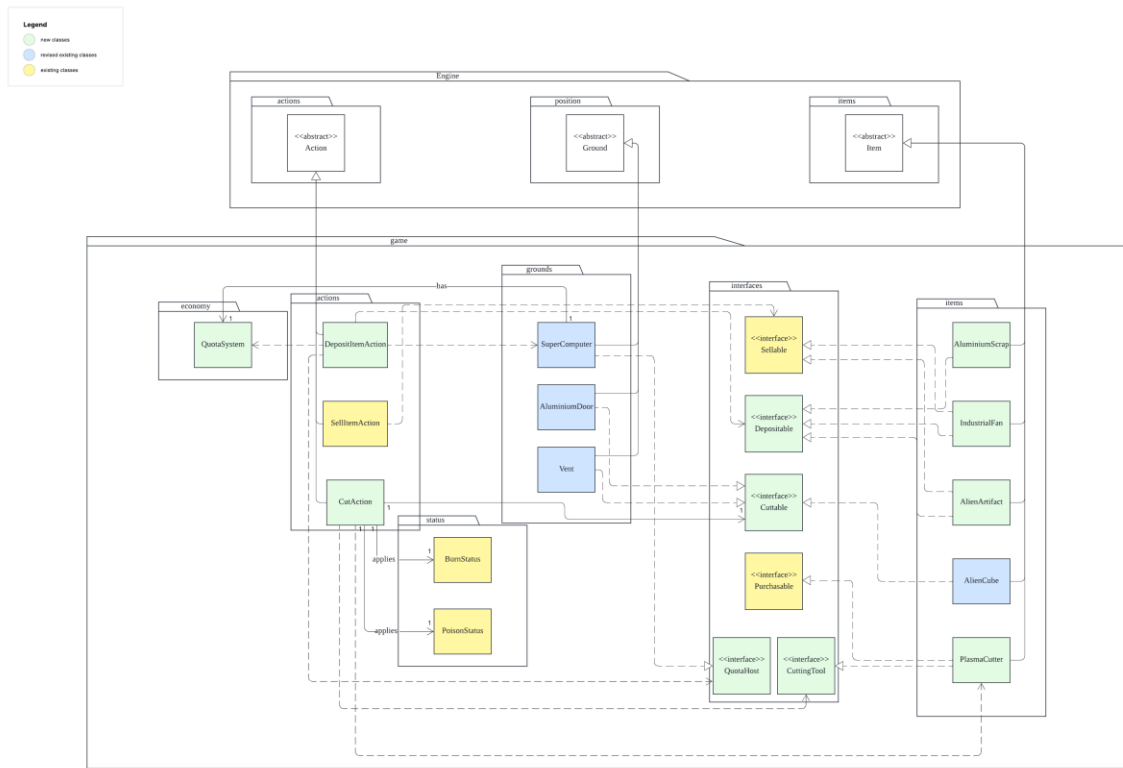


FIT2099 Assignment 3 - REQ1 Design Rationale

REQ1 Design Class Diagram



Design Rationale

1. Separating worker credits from company quota

Design A: Dedicated QuotaSystem + Wallet

Worker spending power lives in Wallet (used by BuyItemAction and SellItemAction), while company quota progress is tracked independently in QuotaSystem. Supercomputer implements QuotaHost and delegates per-turn quota logic to QuotaSystem.tick(), while DepositItemAction calls quotaHost.getQuotaSystem().progressQuota() after a successful deposit.

Pros	Cons
Clear separation of two currencies — worker credits vs company quota —	QuotaSystem must be initialised separately via Supercomputer.initialiseQuotaSystem(Location)

matching the requirement spec. QuotaSystem has a single responsibility: rank, deadline, progress, and firing adjacent workers. Supercomputer does not grow into a god-class handling both shop logic and quota arithmetic. Easy to test quota cycles in isolation (QuotaSystemTest) without spinning up the full game.	after map placement. Slight connascence of position: QuotaSystem stores the supercomputer Location to find adjacent workers.
---	--

Design B: Store Quota Fields Directly on Supercomputer

Embed currentRank, baseQuota, currentTurn, and related fields on Supercomputer and handle all quota logic inside tick() and allowableActions().

Pros	Cons
Fewer classes; all terminal behaviour in one place (KISS at file level). No separate initialisation step.	Violates Single Responsibility Principle — one class owns shop catalogue, buy/sell/deposit actions, and quota lifecycle. Harder to unit test quota rank progression without constructing a full Supercomputer and map. Increases connascence of algorithm — any future terminal type would duplicate quota logic or depend on Supercomputer concretely.

Design C: Single Unified Credit Class

One Credit object tracks both worker balance and company quota progress, with methods such as spendWorker() and depositCompany().

Pros	Cons
One place to look for all money in the game.	Conflates two distinct game concepts that behave differently (wallet cap of 1000 vs unbounded quota progress). Violates Interface Segregation Principle — buy/sell code must know about quota fields it never uses. Code smell: feature envy — deposit logic

	would reach into a shared object rather than updating the subsystem that owns quota state.
--	--

Chosen Design: Design A

A dedicated QuotaSystem paired with the existing Wallet best models the two separate currencies in the requirement, keeps Supercomputer focused on terminal interactions, and allows quota logic to be unit tested independently.

2. Transaction capabilities via interfaces

Design A: Purchasable, Sellable and Depositable Interfaces

Each item declares what it can do by implementing the relevant interfaces. Supercomputer.allowableActions() discovers capabilities polymorphically using item.asCapability(Sellable.class) and item.asCapability(Depositable.class). Item-specific side effects live on the item itself — for example, PlasmaCutter.onPurchased() applies burn damage, IndustrialFan.onSold() spawns a Slime, and AlienArtifact.onDeposit() teleports the worker.

Pros	Cons
Follows Open/Closed Principle — new resources only need new classes implementing the right interfaces; no changes to Supercomputer. Eliminates instanceof checks, a known Assignment 2 code smell markers penalise. BuyItemAction, SellItemAction, and DepositItemAction share transaction boilerplate via TransactionAction (DRY). Sell vs deposit choice is natural: both actions appear in the menu and the player picks one.	IndustrialFan implements three interfaces even though it is never bought — interface count can look noisy. Deposit reward is split: getDepositReward() on the item, but DepositItemAction applies it to QuotaSystem — mild connascence of convention.

Design B: Giant switch in Supercomputer

Supercomputer inspects each inventory item's concrete type and builds the correct action inline.

Pros	Cons
All transaction rules visible in one method.	Major code smell: duplicated type-checking logic; violates OCP — every new item requires editing Supercomputer. Directly repeats the Assignment 2 anti-pattern (if (item instanceof AluminiumScrap)). High connascence of name — Supercomputer must know every concrete item class.

Design C: Separate Action Classes per Item

Create SellIndustrialFanAction, DepositIndustrialFanAction, DepositAluminiumScrapAction, and so on.

Pros	Cons
Each action is small and explicit.	Massive duplication of remove item, apply effect, print message (violates DRY). Adding one resource with sell and deposit requires two new action classes. Does not scale to a shop with many purchasable items.

Chosen Design: Design A

Capability interfaces let each item own its transaction behaviour while shared action classes handle the common flow, removing instanceof checks and keeping Supercomputer closed for modification when new loot types are added.

3. The Cutting System (Cuttable + CuttingTool + CutAction)

Design A: Cuttable Interface with polymorphic onCut()

AluminiumDoor, Vent, and AlienCube each implement Cuttable. ContractedWorker discovers nearby grounds via getGroundAs(Cuttable.class) and inventory items via getItemsAs(Cuttable.class). A single CutAction orchestrates the cut: calls target.onCut(), drops getDroppedItem(), then transforms ground to Floor or removes the inventory item.

Pros	Cons
Each cut target owns its own behaviour — door explosion, Undead spawn, poison on cube. AlienCube.canBeCut() encodes the inventory-only rule without special cases in CutAction. Aligns with Liskov Substitution Principle — any Cuttable can be passed to CutAction. Avoids Assignment 2-style instanceof Door / instanceof Vent branching.	CutAction must handle two target kinds (ground tile vs inventory item), creating mild connascence of execution in post-cut cleanup. CuttingTool is a marker interface with no methods — it documents intent more than behaviour.

Design B: One CutAction per Target Type

CutDoorAction, CutVentAction, and CutAlienCubeAction each with duplicated plasma-cutter checks.

Pros	Cons
Each action class is self-contained.	Duplicated do I have a cutter? logic across three classes (DRY violation). ContractedWorker must know every cuttable type to offer the right actions.

Design C: Cut Logic Entirely Inside PlasmaCutter

PlasmaCutter methods such as cutDoor(), cutVent(), and cutCube() called from worker code.

Pros	Cons
Tool-centric mental model.	Feature envy — PlasmaCutter accumulates knowledge of doors, vents, cubes, explosions, and spawns. Ground classes lose ownership of their cut consequences; harder to extend cutting to a future tool.

Chosen Design: Design A

A single Cuttable interface with one CutAction keeps cutting behaviour on the objects being cut while avoiding duplicated cutter checks and concrete type branching in the worker.

4. Accessing quota without concrete casting (QuotaHost)

DepositItemAction retrieves the quota system via QuotaHost quotaHost = supercomputerLocation.getGroundAs(QuotaHost.class), then calls quotaHost.getQuotaSystem(). This avoids casting to Supercomputer, supporting Dependency Inversion Principle — the action depends on QuotaHost, not a concrete ground class.

Design A: QuotaHost Interface on Supercomputer

Pros	Cons
Actions remain decoupled from Supercomputer; any future QuotaHost ground type would work without changing DepositItemAction.	Requires Supercomputer to implement an extra interface; slightly more indirection than passing QuotaSystem directly.

Design B: Pass QuotaSystem Directly into DepositItemAction

Pros	Cons
Simpler constructor; no interface lookup at execution time.	Couples action creation to a concrete QuotaSystem reference at menu-build time; harder to reuse if another terminal hosts quota.

Chosen Design: Design A

QuotaHost keeps DepositItemAction dependent on an abstraction rather than Supercomputer or a concrete QuotaSystem reference, aligning with the Dependency Inversion Principle.

Conclusion

The dual-economy split (Wallet for worker credits, QuotaSystem for company quota) directly models the assignment specification and keeps each subsystem cohesive.

Extracting QuotaSystem from Supercomputer follows the Single Responsibility Principle and makes quota rank progression, deadline firing, and the 5%/10% ceiling increases testable without the full shop catalogue.

Transaction interfaces replace the Assignment 2 anti-pattern of instance of item checks. Each resource class encapsulates its own side effects while shared transaction flow stays in TransactionAction subclasses. This follows the Open/Closed Principle and DRY.

The cutting system distributes behaviour to the objects being cut, keeping CutAction as a thin orchestrator. Status effects (BurnStatus, PoisonStatus) tick each turn with remaining-duration messages, satisfying the per-turn console output requirement.

Maintainability foresight: new loot types, terminals, or cutting tools can be introduced by adding classes or interfaces without modifying existing quota, transaction, or cut logic.